

MTB Post-Incident Review- Case MTB-IR-2026-0902

Carin Madden
PNC CyberBootcamp 2026
September 10, 2026

Executive Summary

Meridian Trust Bank experienced a cybersecurity incident that began with suspicious activity against the public web server, MTB-WEB01, on August 28, 2026. The attacker gained access to the web server, established persistence, obtained credentials and SSH access, moved to MTB-FILE01, and transferred data out of the environment. The investigation found weaknesses in vulnerability management, network segmentation, credential protection, and security monitoring. Meridian Trust Bank should improve its patching and vulnerability scanning process, restrict access between the DMZ and internal servers, and deploy new detection rules.

What Happened

The first known attacker activity occurred on August 28, 2026 at 1:52:17 AM, when suspicious scanning activity was observed against the public environment. The attacker continued probing the web server and successfully exploited MTB-WEB01 on August 30 at approximately 2:09 AM.

After gaining access, the attacker placed style.php in the web upload directory. The attacker later escalated privileges to root, created the svc-backup account with root privileges, established SSH access, and created a malicious cron job for persistence.

The attacker collected data and used tar to archive it for transfer. Access from MTB-WEB01 was then used to move through SSH to MTB-FILE01. On the file server, additional persistence was established using a systemd service that communicated with an external destination.

Data was eventually transferred out of the environment through outbound network connections. The intrusion did not reach a human analyst until September 2, several days after the first suspicious activity.

ATT&CK Mapping

The intrusion involved multiple MITRE ATT&CK techniques. Reconnaissance included Vulnerability Scanning (T1595.002), while initial access involved Exploit Public Facing Application (T1190). Persistence included Web Shell (T1505.003), Local Account (T1136.001), SSH Authorized Keys (T1098.004), Cron (T1053.003), and Systemd Service (T1543.002).

The attacker also used Exploitation for Privilege Escalation (T1068). Private Keys (T1552.004) were involved in credential access, and SSH (T1021.004) was used for lateral movement. Data was collected using Archive via Utility (T1560.001), while Web Protocols (T1071.001) were used for command and control and Exfiltration Over C2 Channel (T1041) was used to transfer data.

The investigation also identified Clear Command History (T1070.003), Obfuscated Files or Information (T1027), Network Service Discovery (T1046), and File and Directory Discovery (T1083).

Detection Gap

The first known attacker activity occurred on August 28 at 1:52:17 AM, and the first successful exploitation occurred on August 30 at 2:09:12 AM. This provided approximately two days where earlier detection and response could have stopped the intrusion.

The four new detection rules would have provided several opportunities to respond. `mtb_detect_03` would have detected the CGI traversal activity. `mtb_detect_02` would have fired when `style.php` appeared on August 30 at 2:15:22 AM. `mtb_detect_01` would have detected the malicious cron job on September 2 at approximately 2:26 AM. `mtb_detect_04` would have detected the suspicious `svc-backup` SSH login to `MTB-FILE01` at 2:52:14 AM.

`mtb_detect_03` would have made the biggest difference because responding to it could have stopped the attack before the attacker established access to `MTB-WEB01`. The later rules could still have limited persistence, lateral movement, and data loss.

The rules would not detect everything. They could miss initial reconnaissance, credential or private key theft, normal looking C2 traffic, exfiltration, or attacks that do not match the specific patterns being monitored.

New Detections

`mtb_detect_01`: Cron or Systemd Persistence

This rule detects cron or systemd persistence that executes binaries from `/usr/local/`. It targets the **TTP tier** of the Pyramid of Pain because it focuses on how the attacker establishes persistence. To evade it, the attacker would need to change the persistence technique or execution location.

Severity: Critical

`mtb_detect_02`: Script Written to Web Upload Directory

This rule detects script files written into web accessible upload directories. It targets the **TTP tier** because it detects the behavior of placing executable scripts in upload directories instead of only looking for `style.php`. The attacker would need to change how or where the malicious script is placed.

Severity: Critical

`mtb_detect_03`: Encoded CGI Directory Traversal

This rule detects encoded directory traversal against CGI paths. It targets the **TTP tier** because it detects the attack method instead of a specific attacker IP address. The attacker would need to change the exploitation technique or request pattern to avoid detection.

Severity: High

mtb_detect_04: Suspicious SSH Service Account Login

This rule detects successful SSH authentication to a service account from a server segment host. It targets the **TTP tier** because it focuses on suspicious lateral movement behavior. The attacker would need to change the account, source system, protocol, or technique to avoid detection.

Severity: High

Containment, Eradication, and Recovery

The first step should be preserving volatile evidence from MTB-WEB01 and MTB-FILE01, including memory, active processes, network connections, and logged in users. Both systems should then be isolated from the network while remaining powered on so investigators can preserve important evidence.

Known malicious C2 communication should be blocked, and the compromised svc-backup credentials and SSH keys should be disabled or rotated. Every system that trusted the compromised private key should also be reviewed.

On MTB-WEB01, Apache should be updated and the vulnerable CGI configuration corrected. The style.php file should be preserved as evidence and then quarantined. Executable scripts should no longer be allowed to run from the upload directory. The malicious cron job and its associated binary should also be removed.

On MTB-FILE01, unnecessary file permissions should be removed and sensitive directories reviewed. The malicious systemd service should be disabled and removed. Network controls should also restrict unnecessary outbound connections from servers.

MTB-FILE01 should return to service first because it is an internal server and can be restored under tighter controls. MTB-WEB01 should return second because it is exposed to the internet and requires more testing.

A backup from before the successful compromise should be used. The current plan identifies a backup from around August 14 as a safe option. Before restoring it, the backup should be scanned for malicious files, unauthorized accounts, SSH keys, cron jobs, systemd services, and other attacker artifacts.

A 30 day heightened monitoring period should begin after the systems return to service. Authentication, SSH connections, web requests, file creation, cron and systemd changes, outbound connections, and the four new detection rules should be closely monitored.

Root Cause and Contributing Factors

The vulnerability management process allowed an internet facing server with a known vulnerability to remain exposed. Critical vulnerabilities affecting public systems should be prioritized, patched, verified, and tracked until they are fixed.

The September 8 assessment also occurred too late to identify the vulnerability before the August 30 exploitation. Internet facing systems should be scanned at least weekly, with additional scans when important vulnerabilities are announced. More frequent scanning would require additional scanner resources, network resources, analyst time to review findings, and administrator time to test, deploy, and verify patches.

The SSH private key should not have been stored on the public web server because if that server was compromised, the attacker could use the key to access other servers. The DMZ should have very limited access to internal servers and only allow connections that are actually needed.

The environment also recorded repeated outbound connections without creating an alert. Seven and a half hours of connections every five minutes to an unapproved destination appeared in the egress logs, but there was no detection rule looking for that pattern. The data existed, but the monitoring needed to recognize it as suspicious was missing.

Recommendations

Meridian Trust Bank should scan internet facing systems at least weekly and perform additional scans when important vulnerabilities are announced. Critical findings should be assigned to an owner, patched quickly, and verified with another scan.

Access between the DMZ and internal servers should be limited to only what is necessary. SSH keys should be properly protected, and compromised keys should be replaced on every system that trusted them.

The four new Sigma detection rules should be deployed and monitored by the SOC. Another detection should also be created for repeated outbound connections to destinations that are not approved.

Finally, MTB-WEB01 should not return to the internet until its vulnerabilities and Apache configuration have been fixed and verified. Both systems should then be monitored closely for 30 days for signs of the attacker returning.

Google sheets where evidence lives and more documentation occurred:  Week 6 Day 4

Disclaimer: Chatgpt was used to make this report flow easier with wording and formatting.